

SQL Basics

Questions

1. Create a New Database and Table for Employees

Task: Create a new database named **company_db** and Create a table named **employees** with the following columns:

Column Name	Data Type	Constraint
employee_id	INT	PRIMARY KEY
first_name	VARCHAR(50)	
last_name	VARCHAR(50)	
department	VARCHAR(50)	
salary	INT	
hire_date	DATE	

Answer: Here's the SQL code to create the database and the employees table as specified:

-- **Step 1:** Create a new database

```
CREATE DATABASE company_db;
```

-- **Step 2:** Use the new database

```
USE company_db;
```

-- **Step 3:** Create the employees table

```
CREATE TABLE employees (  
    employee_id INT PRIMARY KEY,  
    first_name VARCHAR(50),  
    last_name VARCHAR(50),  
    department VARCHAR(50),
```

```

salary INT,

hire_date DATE

);

```

Explanation:

1. CREATE DATABASE company_db; → Creates a new database named company_db.
2. USE company_db; → Switches to the newly created database.
3. CREATE TABLE employees (...); → Creates the table employees with the columns and constraints specified:
 - employee_id → Integer and primary key
 - first_name, last_name, department → Text with max 50 characters
 - salary → Integer
 - hire_date → Date

This table is now ready to insert employee data.

2. Insert Data into Employees Table

Task: Insert the following sample records into the **employees** table.

employee_id	first_name	last_name	department	salary	hire_date
101	Amit	Sharma	HR	5000	2020-01-15

				00	
102	Riya	Kapoor	Sales	75000	2019-03-22
103	Raj	Mehta	IT	90000	2018-07-11
104	Neha	Verma	IT	85000	2021-09-01
105	Arjun	Singh	Finance	60000	2022-02-10

Answer: Here's the SQL code to insert the given sample records into the employees table:

-- Insert data into employees table

```
INSERT INTO employees (employee_id, first_name, last_name, department, salary, hire_date)
```

```
VALUES
```

```
(101, 'Amit', 'Sharma', 'HR', 50000, '2020-01-15'),
```

```
(102, 'Riya', 'Kapoor', 'Sales', 75000, '2019-03-22'),
```

```
(103, 'Raj', 'Mehta', 'IT', 90000, '2018-07-11'),
```

```
(104, 'Neha', 'Verma', 'IT', 85000, '2021-09-01'),
```

```
(105, 'Arjun', 'Singh', 'Finance', 60000, '2022-02-10');
```

Explanation:

- INSERT INTO employees (...) VALUES (...); adds the records to the employees table.
- Each row corresponds to one employee, with values matching the columns order.

After executing this, the employees table will contain all 5 sample employees.

3. Display All Employee Records Sorted by Salary (Lowest to Highest)

Hint: Use the **ORDER BY** clause on the salary column.

Answer: Here's the SQL query to display all employee records sorted by salary in ascending order:

```
SELECT *  
  
FROM employees  
  
ORDER BY salary ASC;
```

Explanation:

- SELECT * → Retrieves all columns from the employees table.
- ORDER BY salary ASC → Sorts the results by the salary column from lowest to highest.
- You can use DESC instead of ASC to sort highest to lowest if needed.

This query will show all employees ranked by their salary from the lowest-paid to the highest-paid.

4. Show Employees Sorted by Department (A–Z) and Salary (High → Low)

Answer: Here's the SQL query to sort employees first by department alphabetically (A–Z) and then by salary in descending order within each department:

```
SELECT *  
  
FROM employees  
  
ORDER BY department ASC, salary DESC;
```

Explanation:

- ORDER BY department ASC → Sorts employees alphabetically by department.
- salary DESC → Within each department, sorts employees from highest to lowest salary.
- Using multiple columns in ORDER BY allows primary and secondary sorting.

This will display employees grouped by department and ranked by salary within each department.

Based on the sample data provided, if we sort employees by Department (A–Z) and then by Salary (High → Low), the result will be:

employee_id	first_name	last_name	department	salary	hire_date
105	Arjun	Singh	Finance	60000	2022-02-10
101	Amit	Sharma	HR	50000	2020-01-15
103	Raj	Mehta	IT	90000	2018-07-11
104	Neha	Verma	IT	85000	2021-09-01
102	Riya	Kapoor	Sales	75000	2019-03-22

SQL Query to Achieve This:

```
SELECT *  
  
FROM employees  
  
ORDER BY department ASC, salary DESC;
```

Explanation:

- department ASC → Departments sorted alphabetically (A → Z)
- salary DESC → Within each department, employees sorted by highest salary first

This gives a grouped and prioritised view of employees by department and salary.

5. List All Employees in the IT Department, Ordered by Hire Date (Newest First)

Answer: To list all employees in the IT department, ordered by hire date from newest to oldest, the SQL query would be:

```
SELECT *  
FROM employees  
WHERE department = 'IT'  
ORDER BY hire_date DESC;
```

Result Based on Provided Data:

employee_id	first_name	last_name	department	salary	hire_date
104	Neha	Verma	IT	85000	2021-09-01
103	Raj	Mehta	IT	90000	2018-07-11

Explanation:

- WHERE department = 'IT' → Filters only IT employees.
- ORDER BY hire_date DESC → Sorts by hire date newest first.

This shows Neha Verma (newest hire) before Raj Mehta (earlier hire).

6. Create and Populate a Sales Table

Task: Create a table **sales** to track sales data:

sale_id	customer_name	amount	sale_date
1	Aditi	1500	2024-08-01
2	Rohan	2200	2024-08-03
3	Aditi	3500	2024-09-05
4	Meena	2700	2024-09-15
5	Rohan	4500	2024-09-25

Answer: Here's the SQL code to create the sales table and insert the provided records:

-- **Step 1:** Create the sales table

```
CREATE TABLE sales (  
    sale_id INT PRIMARY KEY,  
    customer_name VARCHAR(50),  
    amount INT,  
    sale_date DATE  
);
```

-- **Step 2:** Insert the sample records

```
INSERT INTO sales (sale_id, customer_name, amount, sale_date)
VALUES
(1, 'Aditi', 1500, '2024-08-01'),
(2, 'Rohan', 2200, '2024-08-03'),
(3, 'Aditi', 3500, '2024-09-05'),
(4, 'Meena', 2700, '2024-09-15'),
(5, 'Rohan', 4500, '2024-09-25');
```

Explanation:

1. CREATE TABLE sales (...) → Defines the table with columns:
 - sale_id → Primary key
 - customer_name → Text up to 50 characters
 - amount → Integer for sale amount
 - sale_date → Date of sale
 2. INSERT INTO sales (...) VALUES (...) → Adds the 5 sample sales records to the table.
- After executing this, the sales table is ready for queries.

7. Display All Sales Records Sorted by Amount (Highest → Lowest)

Hint: Use **ORDER BY amount DESC**.

Answer: To display all sales records sorted by amount from highest to lowest, use the following SQL query:

```
SELECT *
FROM sales
ORDER BY amount DESC;
```


Result Based on Provided Data:

sale_id	customer_name	amount	sale_date
5	Rohan	4500	2024-09-25
3	Aditi	3500	2024-09-05
4	Meena	2700	2024-09-15
2	Rohan	2200	2024-08-03
1	Aditi	1500	2024-08-01

Explanation:

- ORDER BY amount DESC → Sorts the sales table by the amount column in descending order, showing highest sales first.

8. Show All Sales Made by Customer “Aditi”

Hint: Use **WHERE customer_name = 'Aditi'**.

Answer: To display all sales records made by customer “Aditi”, the SQL query is:

```
SELECT *
```

```
FROM sales
```

```
WHERE customer_name = 'Aditi';
```

Result Based on Provided Data:

sale_id	customer_name	amount	sale_date
1	Aditi	1500	2024-08-01
3	Aditi	3500	2024-09-05

Explanation:

- WHERE customer_name = 'Aditi' → Filters the records to show only those where the customer is Aditi.
- SELECT * → Retrieves all columns for the matching records.

9. What is the Difference Between a Primary Key and a Foreign Key?

Answer: Here's a clear explanation of the difference between a Primary Key and a Foreign Key in databases:

1. Primary Key (PK)

Definition:

- A primary key is a column (or a set of columns) that uniquely identifies each row in a table.

Key Points:

- Unique: No two rows can have the same primary key value.
- Not Null: Cannot contain NULL values.
- One per table: Each table can have only one primary key.
- Purpose: Ensures entity integrity (uniqueness of records).

Example:

In an employees table:

```
employee_id INT PRIMARY KEY
```

- employee_id uniquely identifies each employee.

2. Foreign Key (FK)

Definition:

- A foreign key is a column (or set of columns) in one table that refers to the primary key in another table.

Key Points:

- Maintains Referential Integrity: Ensures that the value in the foreign key exists in the referenced primary key table.
- Can have duplicates: Multiple rows in the child table can reference the same parent row.
- Can be NULL: Unless specified otherwise.
- Purpose: Establishes a relationship between two tables.

Example:

In a sales table:

customer_id INT,

FOREIGN KEY (customer_id) REFERENCES customers(customer_id)

- customer_id in sales refers to customer_id in customers.

Key Differences

Aspect	Primary Key	Foreign Key
Purpose	Uniquely identifies a record in its own table	References a record in another table
Uniqueness	Must be unique	Can have duplicates
Null Values	Cannot be NULL	Can be NULL (optional)

Number per Table	Only one	Can have multiple
Integrity	Entity integrity	Referential integrity

Summary:

- Primary Key → Uniquely identifies rows in a table.
- Foreign Key → Links two tables and ensures data consistency between them.

10. What Are Constraints in SQL and Why Are They Used?

Answer: Constraints in SQL

Definition:

Constraints are rules applied to table columns to enforce data integrity and accuracy. They ensure that the data entered into a table follows specific conditions.

Purpose of Constraints

1. **Maintain Data Accuracy:** Ensures only valid data is entered (e.g., no negative salaries).
2. **Enforce Relationships:** Maintains referential integrity between tables (e.g., foreign keys).
3. **Prevent Errors:** Avoids duplicate or inconsistent data (e.g., primary key ensures uniqueness).
4. **Control Data Entry:** Restricts the type, range, or format of data.

Common Types of Constraints

Constraint	Description	Example
PRIMARY KEY	Uniquely identifies each row; cannot be NULL	employee_id INT PRIMARY KEY
FOREIGN KEY	Ensures a column value exists in another table	FOREIGN KEY (customer_id) REFERENCES customers(customer_id)
UNIQUE	Ensures all values in a column are distinct	email VARCHAR(50) UNIQUE
NOT NULL	Prevents NULL values in a column	first_name VARCHAR(50) NOT NULL
CHECK	Enforces a condition on values	salary INT CHECK(salary > 0)
DEFAULT	Sets a default value if none is provided	status VARCHAR(10) DEFAULT 'Active'

Why Use Constraints?

- To guarantee data integrity and correctness.
- To reduce the risk of data anomalies.
- To automate rules enforcement without relying solely on application logic.

Summary:

Constraints are rules applied to table columns that ensure valid, consistent, and reliable data in a SQL database. They are essential for database integrity and proper relational structure.